NC-Archaeologist: Recovering Machined Geometry and Group-Technology Part Families from Legacy CNC Programs

Sanjit Patibanda
School of Mechanical Engineering
Vellore Institute of Technology
Chennai, India
sanjit.patibanda2024@vitstudent.ac.in

***Abstract*—Machine shops accumulate thousands of CNC part programs whose CAD models have been lost, leaving archives that cannot be searched, reused or analysed for Group Technology (GT). We present NC-Archaeologist, a system that works from the G-code alone. It (i) parses programs across controller dialects, (ii) recovers the machined part by 2.5D dexel simulation of material removal, (iii) infers missing cutter diameters from toolpath geometry, (iv) recognises machining features and builds a process sheet, and (v) forms GT part families from a rotation-invariant fingerprint of the recovered geometry and process. On a benchmark of 600 programs from 8 part families written in four dialects with random fixturing, the recovered parts reach a mean removed-volume IoU of 0.925 against design intent. Family discovery reaches an adjusted Rand index of 0.797 and retrieval reaches precision@5 of 0.939. Both are well above text-based baselines (ARI 0.074). On 15 open-source LinuxCNC programs not produced by our generator, 13 are analysed, including parametric and macro programs, and recognisable parts are recovered. To the best of our knowledge, this is the first approach that forms GT part families directly from subtractive CNC programs.**

***Index Terms*—computer-integrated manufacturing, G-code, group technology, reverse engineering, dexel simulation, CAPP, part family formation**

I. Introduction

Computer-integrated manufacturing (CIM) rests on the flow of information between CAD, process planning (CAPP), CAM and the CNC controller [1]–[3]. Industry 4.0 and cyber-physical production systems extend this integration to the whole factory [4], [5]. In practice, however, the flow often runs one way. The NC program is the artefact that survives on the shop floor, while the CAD model and the planner's intent are lost when systems are migrated, suppliers change or programmers retire. A shop may hold thousands of programs named like `O1234.nc` that nobody can classify. This blocks three classic CIM benefits:

- **Reuse:** recognising that a new part resembles one already machined, so that a proven program can be adapted;
- **Group Technology:** forming part families for cell design and standardised set-ups [6], [7];
- **Digital transformation:** turning an opaque archive into searchable data.

Classical GT coding needs a drawing or CAD model, and production-flow analysis needs route sheets [7]. Neither may exist for legacy parts. The G-code text itself is a poor basis for comparison. The same part can be posted with different units, absolute or incremental moves, canned cycles or explicit pecks, arcs or chords, and any rotation or work offset. Two programs for one part can therefore share almost no text.

We instead treat each program as an executable description of geometry. We replay it, recover what it makes, and compare programs by that result. This paper contributes:

1) a dialect-tolerant G-code interpreter that also executes parametric programs (variables, expressions, loops, subroutines, polar coordinates), with explicit warnings for constructs it cannot execute faithfully (cutter compensation, external subprograms, lathe code);
2) recovery of the machined part by dexel simulation, including a *raster-run* heuristic that infers cutter diameters missing from programs without tool comments, and a stock-footprint estimate from the facing toolpath;
3) multi-level, nesting-aware recognition of holes, pockets, slots, steps and island pockets on the recovered part, plus an automatically generated process sheet;
4) a rotation- and translation-invariant fingerprint combining recovered geometry with process content, used for GT family discovery and similarity search;
5) a benchmark of 600 programs in four controller dialects with ground truth, plus hand-written programs that exercise vendor-specific constructs.

II. Related Work

Reverse engineering from toolpaths. Recovering geometry from G-code has been studied for *additive* manufacturing. Tsoutsos *et al.* reconstruct printed solids from toolpaths to detect malicious design changes [8]. Kiński and Sobieski rebuild surfaces from printer G-code through point sets and Poisson reconstruction, motivated by lost solid models [9]. In additive toolpaths the deposited material *is* the part. In milling the part is what *remains* after the cutter has removed material. So the reverse problem requires simulating removal with the correct cutter geometry, which these methods do not address.

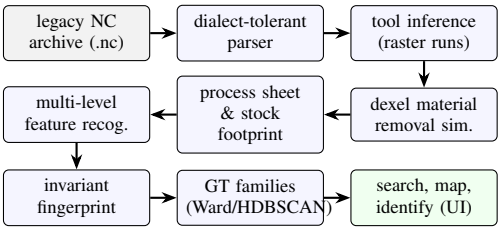


Fig. 1. NC-Archaeologist pipeline. Only the G-code is used as input.

Group Technology and part families. GT classification uses coding systems such as Opitz [6], production-flow analysis [7], and a large literature on cell-formation algorithms [10], [11], from rank-order clustering of machine–part incidence matrices [12] onwards. All of these assume design attributes or routing data as input.

Feature recognition and plan reuse. Machining-feature recognition from B-rep solids is mature [13], retrieval and generative CAPP systems build process plans from part descriptions [14], and CAD-model retrieval has been used to reuse process plans [15]. Neutral product data (STEP, ISO 10303 [16]) and open, interoperable CNC architectures [17] aim to keep this information connected. All of these require the CAD model, which is exactly what is missing for legacy programs.

Learning on G-code. Recent work uses large language models to *generate* G-code [18], and notes that G-code carries no semantic description of the geometry it produces. We address the inverse problem: recovering that geometry and using it.

Simulation. The dexel (depth-element) representation for NC verification goes back to van Hook [19], and discrete NC verification methods for detecting machining errors are well established [20]. Simulation models of this kind are also the basis of manufacturing digital twins [21]. We use the height-map form as a fast, adequate model for 3-axis prismatic parts.

III. Method

Fig. 1 shows the pipeline.

A. Dialect-tolerant interpretation

A tokenizer and modal state machine follow ISO 6983 [22]. They handle motion modes G0–G3 (IJK and R arcs, helical moves), units G20/G21, distance modes G90/G91, canned drilling cycles G81–G83 with G98/G99 retract, tool changes and work offsets. Arcs are linearised to a 0.05 mm chord tolerance. Fanuc subprograms (`M98`/`O...M99`) defined in the same file are inlined. Parametric programs are executed by a macro layer before interpretation: numbered and named parameters, bracket expressions with arithmetic, comparison and trigonometric functions, LinuxCNC `o`-word subroutines with arguments, `if`/`while`/`repeat`, and Fanuc Macro B `WHILE...DO`/`END` and `IF...GOTO`. Polar coordinates (`@` radius, `^` angle) and repeat counts `L` on canned cycles are supported. Constructs the interpreter cannot execute faithfully are *flagged rather than guessed*: controller-side cutter compensation G41/G42, calls to subprograms in other files, non-XY planes, several work offsets in one program (fixture stations drawn in one frame) and lathe programs. Each program carries these warnings into the user interface.

B. Cutter inference

Diameters are read from tool comments when present. Many legacy programs have no comments, so we infer the diameter from geometry. Inside a *raster run* (at least three alternating, parallel, equal-Z passes joined by short link moves with no rapid in between), the pass spacing equals the step-over s. Requiring this pattern rejects look-alikes such as separate parallel slots or the two long sides of a profile loop. With a typical step-over ratio of 0.5, $\hat{D} = \mathrm{snap}(s/0.5)$ to the nearest standard cutter. A single shallow level of long passes is treated as facing, with ratio 0.7. Drills and non-raster tools fall back to a default diameter, and the program is flagged.

C. Material-removal simulation

The stock is a height map Z_{ij} (default 0.25 mm cells) over the XY extent of all cutting moves. The stock top is $Z{=}0$, the most common convention. Each feed move lowers Z by the swept cutter envelope:

$$Z_{ij} \leftarrow \min\big(Z_{ij},\ z(t^*) + h(r_{ij})\big), \tag{1}$$

where h is the cutter profile (flat, ball, 118° drill point, or chamfer). For flat cutters the minimum over the move is evaluated in closed form, using the end points of the covered parameter interval. Shaped cutters are sampled along the move. Per-tool-block removal masks feed the process analysis.

D. Process sheet and stock footprint

The program is split into tool blocks. Each block is classified from what it removed as facing, pocketing, island pocketing, bore, slotting, step milling, profiling or drilling. The rules use depth, coverage, enclosure (filled-hole area relative to cut area), elongation relative to cutter width, and contact with the stock edge. Cycle time is estimated from feed rates, rapid traverse, tool changes and pecks. G-code does not state the stock size, so we estimate the stock footprint from the facing toolpath. Facing passes overrun the stock by about one cutter radius at their ends and overhang the side edges by about half a radius, so the pass envelope is contracted along and grown across the passes accordingly.

E. Multi-level feature recognition

Flat-floor depths (cells with near-zero slope, so drill cones and scallops are ignored) define plateau levels. Connected regions are extracted between successive levels after morphological smoothing. Each region becomes a feature whose parent is the region containing it one level up, and a region identical to its parent is not repeated. Regions are labelled as hole, circular pocket, pocket, slot, step, island pocket or outline. The labels use circularity, a fill ratio that separates circles (about 0.785) from rounded squares (about 1), principal-axis aspect ratio,

enclosed uncut area and stock-edge contact. Hole patterns are tested for a common pitch circle diameter.

F. Fingerprint and family discovery

A fingerprint should describe *what* a program makes, independent of how it is written or fixtured. The geometric block is computed on a design-depth image from which facing, the outer profiling ring and cuts in air outside the estimated stock are removed. It contains the depth histogram relative to the deepest cut, the seven Hu moment invariants [23], a radial depth profile normalised by the radius of gyration, log feature counts, elongation, normalised hole size, bolt-circle flag and nesting. All of these are invariant to rotation and translation. The process block contains the operation-time mix, operation presence and counts, and tool statistics. Each block is standardised, weighted (0.5/0.5, chosen on a separate development set) and normalised by its dimension. Families come from Ward agglomerative clustering [24], with k chosen by the silhouette coefficient [25] when unknown. HDBSCAN [26], [27] provides density-based families and flags unusual programs. Similarity search uses cosine k-NN, and the archive map uses t-SNE [28].

G. Implementation and interactive dashboard

The system is written in Python (about 3,800 lines, using NumPy, SciPy and scikit-learn [29]; region perimeters and Hu moments are implemented directly) with 22 unit tests for the interpreter, the macro layer and the simulator. A FastAPI back end serves a browser dashboard (Fig. 2) built with Three.js and Plotly. Users can browse the archive map and the discovered families, inspect a recovered part in 3D, and replay its machining: the replay lowers the height map move by move and shows the G-code line being executed. The dashboard also shows the process sheet, recognised features and similar programs, and explains every G-code line in plain language. Any new program can be uploaded and is analysed and assigned to a family in about two seconds, which makes the method usable as an everyday archive tool rather than an offline experiment.

IV. Benchmark

No public archive of CNC programs with part-family labels exists, so we built a synthetic benchmark with a fully known ground truth. We state this openly and complement it with hand-written programs (Section V-F).

- **Eight parametric families**: bolt-hole plate, pocket housing, slotted bracket, flange, stepped block, multi-pocket tray, keyway bar and boss (island in pocket). Dimensions, counts, depths, cutters and optional facing and profiling are randomised.
- **A mini CAM**: zig-zag or offset-spiral pocketing, concentric bores, multi-level slots, shoulder rasters, peck or canned drilling, and climb or conventional profiles. Step-over is 40–60 % and step-down 1.5–3.5 mm.
- **Four post-processor dialects**: Fanuc-like with N-numbers, IJ arcs and G83; Haas-like with R arcs and G81/G83 cycles; *minimal* with no comments, chord-linearised arcs and explicit pecks; and inch + incremental (G20/G91).
- **Random fixturing**: rotation (half at 0°, a quarter at multiples of 90°, a quarter arbitrary) and work zero at the stock centre or corner.

File names carry no family information. Ground truth (family, design spec and an analytic design-depth function) is used only for evaluation. All parameters were tuned on a development set (seed 7). All reported results are on a held-out test set of 600 programs (seed 11, 75 per family).

TABLE I
Recovered part vs. design intent (test set)

	all	tools in comments	tools inferred
programs	600	448	152
removed-volume IoU	0.925	0.999	0.709
RMS depth error (mm)	1.17	0.19	4.06
median RMS (mm)	0.20	0.11	3.82
hole recall	100.0%	100.0%	100.0%
hole precision	99.9%	99.8%	100.0%
hole ⌀ MAE (mm)	0.37	0.01	1.44

TABLE II
Feature recognition (count-matched, test set)

feature	# in ground truth	precision	recall
circular pocket	75	1.000	0.987
hole	1542	0.999	1.000
outline	319	1.000	0.962
pocket	582	0.974	0.909
pocket island	75	0.904	1.000
slot	262	0.834	1.000
step	187	0.955	0.797

V. Results

A. Part recovery

Table I compares recovered depth maps with the analytic design intent inside the physical stock. Programs whose tools are all named in comments are recovered almost exactly. Programs that rely on inferred or default cutters (all from the comment-free dialect) lose accuracy wherever a diameter falls back to the default: drills, single-pass slots and profile passes. The skewed distribution (mean vs. median RMS) reflects these cases. Hole positions are recovered to within 0.18 mm on average. Mean runtime is 1.72 s per program (95th percentile 3.92 s) on a laptop CPU.

Table II reports count-based precision and recall of feature recognition per type. Fig. 3 shows an example.

B. Cutter inference

On the comment-free dialect, raster-run inference covered 55.5% of the 339 milling tools. Of these, 43.1% were identified exactly and 81.9% were within one standard size. The remainder (single-pass slots, spiral-only pockets and profiles) fell back to the default diameter and were flagged to the user.

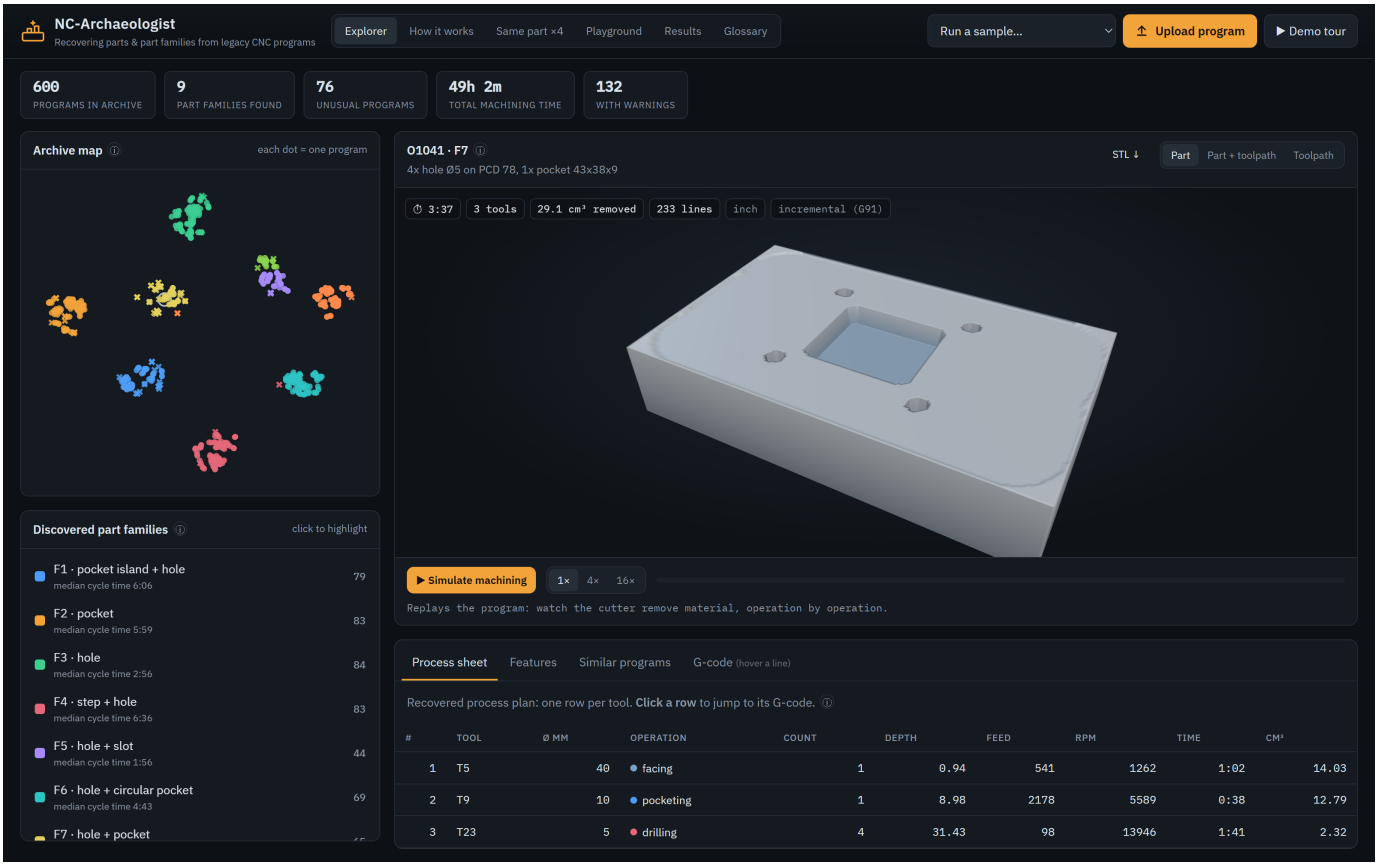


Fig. 2. NC-Archaeologist dashboard on the 600-program test archive: fingerprint map coloured by discovered family (left), automatically named families, a part recovered from an inch/incremental program with machining replay, and its recovered process sheet.

C. Part-family discovery and retrieval

Table III and Fig. 5 compare the full fingerprint with four baselines. Text TF-IDF term weighting [30] works either on raw character n-grams or on the sequence of G/M/T codes with coordinates removed. Clusterings are scored against the true families with the adjusted Rand index (ARI) [31] and normalised mutual information (NMI) [32]. The process-only and geometry-only variants use one block of our fingerprint each. Text baselines mostly recover the posting *dialect* rather than the part family: their clusters agree with the dialect labels at ARI 0.609 (raw) and 0.584 (codes), but with the true families at only 0.074 and 0.055. Combining recovered geometry with process content gives the best results. Without knowing k, the silhouette criterion selects k=9 (ARI 0.770). HDBSCAN finds 10 dense families (ARI 0.815 on inliers) and sets aside 12.7% of programs as unusual.

TABLE III
FAMILY DISCOVERY (k=8, WARD) AND RETRIEVAL

representation	ARI	NMI	purity	P@5	mAP
text TF-IDF (raw)	0.074	0.197	0.232	0.558	0.205
text TF-IDF (codes)	0.055	0.177	0.230	0.580	0.237
process only	0.572	0.668	0.695	0.922	0.746
geometry only	0.618	0.720	0.777	0.934	0.774
full (ours)	**0.797**	**0.822**	**0.907**	**0.939**	**0.851**

D. Dialect invariance

Of the top-5 neighbours, 74.8% come from a *different* dialect than the query, and 94.0% of those belong to the correct family (Fig. 6). In a controlled test, each of 40 parts was posted in all four dialects with independent fixturing (160 programs). The mean cosine similarity between versions of the *same part* was 0.921, against 0.500 for different parts of the same family and -0.083 across families. The textual feature descriptions were character-identical across all four dialects for 25.0% of parts. The remaining differences are in dimensional detail, chiefly drill diameters that the comment-free dialect cannot state and 1–2 mm differences in reported pocket extents, while feature types and counts agree (Fig. 4).

E. Resolution trade-off

On a random sample of 48 programs, the RMS depth error is 0.69, 0.69 and 0.70 mm at 0.5, 0.25 and 0.1 mm cells, while runtime grows from 0.39 to 1.75 and 4.95 s per program. The error is therefore dominated by modelling factors (cutter inference, stock estimate) rather than by grid size. Hole diameter error falls slightly with finer cells (0.27 to 0.24 mm).

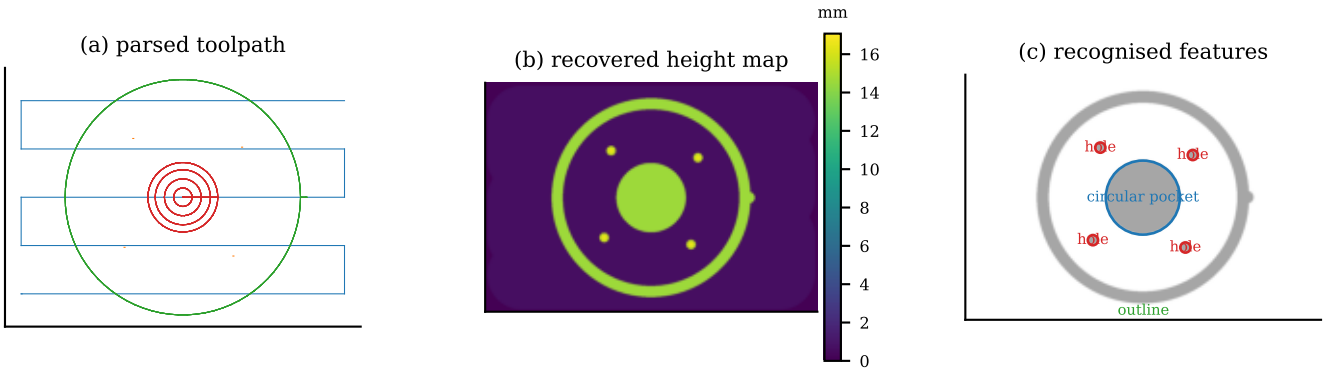


Fig. 3. A flange program (Haas dialect): (a) parsed toolpath, (b) recovered height map, (c) recognised features and bolt circle.

fanuc

```
(T6 - 50.0 MM FACE MILL)
N30 S1284 M03
N40 G54
N50 G43 H6
N60 M08
N70 G0 X91.838 Y-28.195 Z5.
N80 G0 Z2.
N90 G1 Z-0.717 F174
N100 G1 X116.878 Y146.318 F499
```

haas

```
T5 M06 (D50 FACE MILL)
S1284 M03
G54
G43 H5
M08
G00 X-39.5990 Y85.6650 Z5.0000
G00 Z2.0000
G01 Z-0.7170 F174
G01 Y-90.6350 F499
```

minimal

```
Z2.0
G1 Z-0.717 F174
X71.77 Y67.5 F499
X45.36 Y77.191
X-15.374 Y-88.317
X-41.785 Y-78.626
X18.949 Y86.883
X-7.461 Y96.574
X-68.195 Y-68.934
```

inch+inc

```
S1284 M03
M08
G0 X-3.4394 Y-1.5586 Z0.1969
G91
G0 Z-0.1182
G1 Z-0.1069 F6.8504
G1 X6.9409 F19.6457
G1 Y1.1076
G1 X-6.9409
```

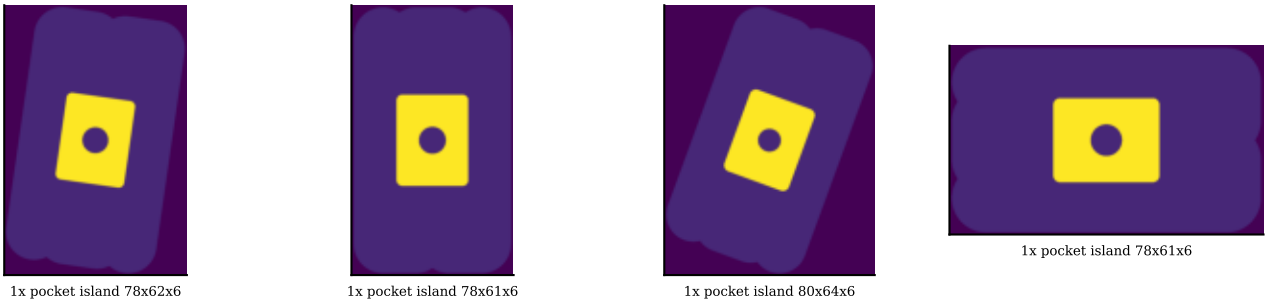


Fig. 4. One part posted in four dialects with independent fixturing. The text differs completely, while the recovered parts and recognised features agree.

We keep 0.25 mm as a compromise for feature measurement; 0.5 mm would suffice for family discovery alone.

F. Hand-written programs

Table V lists hand-written programs that were not produced by our generator. They exercise vendor constructs: cutter compensation, a Haas subprogram, a Fanuc Macro B bolt circle, a helical entry without tool comments, LinuxCNC o-word subroutines, and a lathe program. The subprogram and the Macro B loop are executed and recovered correctly (six holes on the programmed 80 mm pitch circle), and cutter compensation and the lathe program are flagged. One result is wrong in an instructive way: without a facing pass the stock edge is unknown, so the two LinuxCNC window pockets, which lie at the edge of the machined area, are labelled as steps.

G. Open-source real programs

To check the interpreter beyond our own generator, we ran 15 programs from the `nc_files` folder of the LinuxCNC project (GPL-2.0), selected as all 3-axis milling examples plus one lathe program, unmodified. They include a 3D carving written entirely with named parameters (4,700 lines), a stepper-motor mount from a workshop machine using macros and three fixture stations, bolt circles in polar coordinates, cutter-compensation demonstrations and an interpreter torture test. 13 programs are analysed; the other two are not part programs (a remap subroutine and an empty template) and are correctly rejected for having no cutting moves. The macro

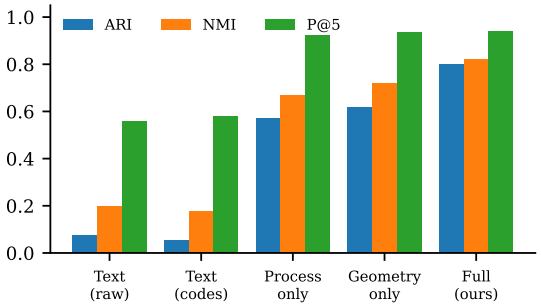


Fig. 5. Ablation: clustering (ARI, NMI) and retrieval (P@5).

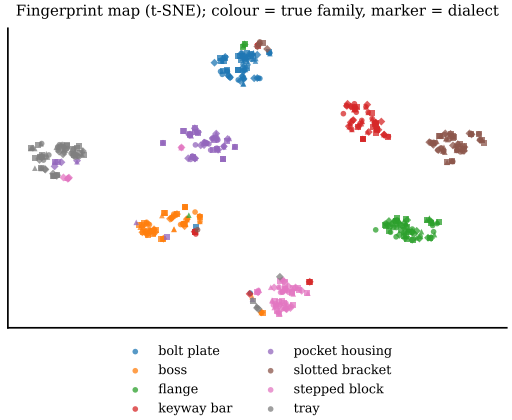


Fig. 6. t-SNE map of fingerprints. Colours are true families and markers are dialects. Dialects mix within each family.

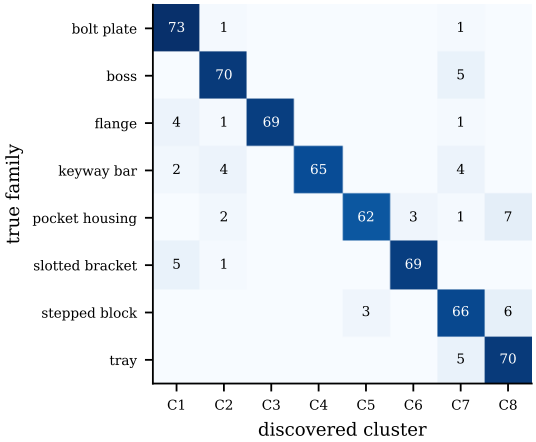


Fig. 7. True families vs. discovered clusters (Hungarian-matched).

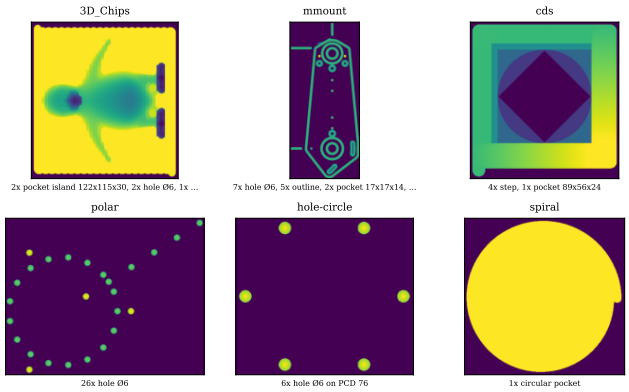


Fig. 8. Parts recovered from unmodified LinuxCNC example programs (depth maps, darker = higher).

layer is needed in 5 of them. These files have no ground truth, so Table IV compares the result with what each file's own comments say it makes. Hole counts and pitch circles match the comments exactly (26 holes in `polar`; six holes on PCD 76 mm for a programmed 3-inch circle), and the carving and motor mount are recognisable (Fig. 8). Drill and cutter sizes are rarely stated in these files, so the default diameter is used and flagged, which limits dimensional accuracy.

VI. Discussion and Limitations

- **Synthetic benchmark.** Families, CAM strategies and dialects were designed by us. The dialect variety and fixturing randomisation make text matching fail, but real archives contain more diverse strategies (adaptive clearing, trochoidal slots). The open-source programs of Section V-G show that the interpreter and reconstruction carry over to real files, but they have no family labels. Quantitative validation on a labelled shop archive is the most important next step.
- **2.5D model.** Height maps cannot represent undercuts, and the system targets 3-axis prismatic parts.
- **Unknown stock and cutters.** Stock size is inferred from facing when present. Drill diameters cannot be inferred from geometry. Cutter compensation is flagged rather than resolved.
- **Not executed.** Subprograms stored in other files, system parameters of a specific controller and cutter compensation are flagged rather than executed. A program using several work offsets is drawn in one frame.

VII. Future Work

Natural extensions are STEP-NC (ISO 14649 [33]) export of the recovered features, so that legacy parts re-enter a modern CAD/CAPP/CAM chain; 5-axis toolpaths with tri-dexel models; learned fingerprints trained on larger archives; and linking families to cell formation and shop-floor scheduling.

VIII. Conclusion

NC-Archaeologist shows that a legacy CNC program contains enough information to recover its part, plan and family. By comparing programs on what they *make* rather than how they are *written*, it groups parts across controller dialects and fixturing, a setting where text similarity fails. This offers a

TABLE IV
OPEN-SOURCE LINUXCNC PROGRAMS (SELECTED)

program	stated in the file	recovered
`3D_Chips`	3D carving of a 100x100x50 mm block with a 10 mm ball-nose cutter	2x pocket island 122x115x30, 2x hole ∅6, 1x po… *macro executed*
`cds`	'Circle Diamond Square' test part on 4x4x2 in stock (stock top at Z=2 in)	4x step, 1x pocket 89x56x24
`comp-g1`	cutter-compensation demo, tool defined only in the controller's tool table	no features below faced surface *cutter comp. flagged*
`g881`	remap subroutine for a custom G81 (not a part program)	rejected: no cutting moves
`hole-circle`	6 bolt holes on a 3 in (76.2 mm) circle, polar coordinates + L repeat	6x hole ∅6 on PCD 76
`lathe_pawn`	lathe program (chess pawn): should be flagged, not milled	1x pocket 27x19x38, 1x pocket 19x19x38 *lathe flagged*
`mmount`	stepper-motor mount, multi-station fixture (G55-G57), macros	7x hole ∅6, 5x outline, 2x pocket 17x17x14, 1x… *macro executed; cutter comp. flagged; multi-station flagged*
`polar`	3 + 1 + 5 + 17 = 26 holes using polar coordinates	26x hole ∅6 *macro executed*
`spiral`	spiral engraving, o-word repeat loop of 800 polar steps	1x circular pocket *macro executed*

TABLE V
HAND-WRITTEN TEST PROGRAMS (QUALITATIVE)

program	recovered description and warnings
`fanuc_cutter_comp` (32 lines)	4x hole ∅9, 1x outline *warnings: Controller-side cutter compensation*
`haas_subprogram` (44 lines)	4x pocket 20x20x2 *warnings: none*
`helical_no_comments` (32 lines)	4x hole ∅6, 1x pocket 38x28x3 *warnings: T8*
`lathe_program` (14 lines)	1x slot 36x6x25 *warnings: Looks like a lathe; Non-XY plane*
`linuxcnc_style` (35 lines)	2x step *warnings: none*
`macro_bolt_circle` (24 lines)	6x hole ∅7 on PCD 80, 1x circular pocket *warnings: none*

practical route to Group Technology and reuse for shops whose only record of a part is its G-code.

REFERENCES

[1] M. P. Groover, *Automation, Production Systems, and Computer-Integrated Manufacturing*, 5th ed. Pearson, 2019.

[2] X. Xu, *Integrating Advanced Computer-Aided Design, Manufacturing, and Numerical Control: Principles and Implementations*. Hershey, PA: IGI Global, 2009.

[3] X. W. Xu and Q. He, "Striving for a total integration of CAD, CAPP, CAM and CNC," *Robotics and Computer-Integrated Manufacturing*, vol. 20, no. 2, pp. 101–109, 2004.

[4] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for Industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, pp. 18–23, 2015.

[5] Y. Lu, "Industry 4.0: A survey on technologies, applications and open research issues," *J. Industrial Information Integration*, vol. 6, pp. 1–10, 2017.

[6] H. Opitz, *A Classification System to Describe Workpieces*. Oxford: Pergamon Press, 1970.

[7] J. L. Burbidge, *Production Flow Analysis for Planning Group Technology*. Oxford: Clarendon Press, 1989.

[8] N. G. Tsoutsos, H. Gamil, and M. Maniatakos, "Secure 3D printing: Reconstructing and validating solid geometries using toolpath reverse engineering," in *Proc. 3rd ACM Workshop on Cyber-Physical System Security (CPSS)*, 2017.

[9] W. Kiński and W. Sobieski, "Geometry extraction from GCODE files destined for 3D printers," *Technical Sciences*, vol. 23, no. 2, pp. 115–130, 2020, doi: 10.31648/ts.5644.

[10] H. M. Selim, R. G. Askin, and A. J. Vakharia, "Cell formation in group technology: review, evaluation and directions for future research," *Computers & Industrial Engineering*, vol. 34, no. 1, pp. 3–20, 1998.

[11] A. Kusiak, "The generalized group technology concept," *International Journal of Production Research*, vol. 25, no. 4, pp. 561–569, 1987.

[12] J. R. King, "Machine-component grouping in production flow analysis: an approach using a rank order clustering algorithm," *International Journal of Production Research*, vol. 18, no. 2, pp. 213–232, 1980.

[13] J. Han, M. Pratt, and W. C. Regli, "Manufacturing feature recognition from solid models: a status report," *IEEE Trans. Robotics and Automation*, vol. 16, no. 6, pp. 782–796, 2000.

[14] H.-C. Zhang and L. Alting, *Computerized Manufacturing Process Planning Systems*. London: Chapman & Hall, 1994.

[15] C. Pitta and M. M. Marefat, "Intelligent retrieval and reuse of CAD solid models," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2005.

[16] ISO 10303-1:1994, *Industrial automation systems and integration — Product data representation and exchange — Part 1: Overview and fundamental principles*.

[17] X. W. Xu and S. T. Newman, "Making CNC machine tools more open, interoperable and intelligent—a review of the technologies," *Computers in Industry*, vol. 57, no. 2, pp. 141–152, 2006.

[18] M. Abdelaal, S. Lokadjaja, and G. Engert, "GLLM: Self-corrective G-code generation using large language models with user feedback," in *Proc. BTW 2025 (Industrial Track)*, 2025, arXiv:2501.17584.

[19] T. Van Hook, "Real-time shaded NC milling display," *ACM SIGGRAPH Computer Graphics*, vol. 20, no. 4, pp. 15–20, 1986.

[20] R. B. Jerard, R. L. Drysdale, K. Hauck, B. Schaudt, and J. Magewick, "Methods for detecting errors in numerically controlled machining of sculptured surfaces," *IEEE Computer Graphics and Applications*, vol. 9, no. 1, pp. 26–39, 1989.

[21] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.

[22] ISO 6983-1:2009, *Automation systems and integration — Numerical control of machines — Program format and definitions of address words*.

[23] M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Trans. Information Theory*, vol. 8, no. 2, pp. 179–187, 1962.

[24] J. H. Ward, "Hierarchical grouping to optimize an objective function," *J. American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.

[25] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *J. Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[26] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. PAKDD*, LNCS 7819, 2013, pp. 160–172.

[27] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *J. Open Source Software*, vol. 2, no. 11, p. 205, 2017.

[28] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[29] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[30] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[31] L. Hubert and P. Arabie, "Comparing partitions," *J. Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[32] A. Strehl and J. Ghosh, "Cluster ensembles—a knowledge reuse framework for combining multiple partitions," *J. Machine Learning Research*, vol. 3, pp. 583–617, 2002.

[33] ISO 14649-1:2003, *Industrial automation systems and integration — Physical device control — Data model for computerized numerical controllers*.